

Letter	Principle	Simple Meaning	Exam Example / Case Study
S	Single Responsibility (SRP)	A class should only do ONE job.	<i>Bad:</i> A class calculates payroll, logs errors, AND sends emails. <i>Good:</i> Split it into a Calculator class, Logger class, and Emailer class.
O	Open/Closed (OCP)	Open for extension (adding new stuff), Closed for modification (changing old code).	<i>Bad:</i> Editing the main <code>Discount</code> file every time you add a new sale. <i>Good:</i> Create a generic <code>Discount</code> class, and make new sub-classes for <code>FlatRate</code> or <code>Percentage</code> .
L	Liskov Substitution (LSP)	A child class must be able to replace its parent class without crashing the program.	<i>Bad:</i> <code>Ostrich</code> inherits from <code>Bird</code> . The program calls <code>fly()</code> , but ostriches can't fly, so it crashes. <i>Good:</i> Both inherit from a general <code>Shape</code> or <code>Animal</code> class with a <code>move()</code> function.
I	Interface Segregation (ISP)	Don't force code to depend on methods it doesn't use.	<i>Bad:</i> A basic printer is forced to inherit <code>scan()</code> and <code>fax()</code> methods it doesn't have. <i>Good:</i> Split into <code>IPrinter</code> , <code>IScanner</code> , <code>IFax</code> .
D	Dependency Inversion (DIP)	High-level code shouldn't depend on low-level details. Both should depend on abstractions (Interfaces).	<i>Bad:</i> Your system is hardcoded to <code>MySQLDatabase</code> . If you switch to <code>MongoDB</code> , everything breaks. <i>Good:</i> Depend on a generic <code>DatabaseInterface</code> .